Publisher and Subscriber

Thumrongsak Kosiyatrakul

Publisher

- A **publisher** publishes data to topics
- A publisher has to have a name (for debugging purpose)
- A topic is a **named** bus (need a name as well)
- A topic must have a type associated with it (strongly typed)
- Do not need to care who are subscribers (anonymous)

talker.py

```
#!/usr/bin/env python

import rospy
from std_msgs.msg import String

def talker():
    pub = rospy.Publisher('chatter', String, queue_size=1)
    rospy.init_node('talker', anonymous=True)
    rate = rospy.Rate(10) # 10hz
    while not rospy.is_shutdown():
        hello_str = "hello world %s" % rospy.get_time()
        rospy.loginfo(hello_str)
        pub.publish(hello_str)
        rate.sleep()

if __name__ == '__main__':
    try:
        talker()
    except rospy.ROSInterruptException:
        pass
```

Do not forget to make this file executable (`chmod +x talker.py`)

- You only need to set it once

talker.py

```
#!/usr/bin/env python
```

- If you try to create a node using Python, the above line must be at the top
- It let the OS knows that this program should be passed to the Python interpreter

```
import rospy
```

- This line should be in all Pyhton node
- It imports all of the basic functionality that we will need

```
from std_msgs.msg import String
```

- We are going to publish messages of type `String`
- We need to import `String` from the standard message `std_msg.msg`.

talker.py

```
pub = rospy.Publisher('chatter', String, queue_size=1)
```

- This create an object of type `Publisher` and can be referred to by the variable named `pub`
- `rospy.Publisher()` is a constructor:
  - **First Argument**: The topic name (`chatter`)
  - **Second Argument**: The type of message (`String`)
  - **Third Argument**: The maximum number of message that will be stored in case any subscriber is not receiving them fast enough

```
rospy.init_node('talker', anonymous=True)
```

- Initialize this node with the name `talker`
- Should be anonymous

talker.py

```
rate = rospy.Rate(10) # 10hz
```

- Create an object of type `Rate` and can be referred to by the variable named `rate`
- `rospy.Rate()` is a constructor
  - The argument 10 specifies the frequency in Hertz
- `rate.sleep()` will be used later which will put this node to sleep for 1/frequency second (0.1 second in this case).

```
while not rospy.is_shutdown():
```

- A loop is a fairly standard `rospy` construct.
- This node should stop if it gets a signal to shutdown.
- `rospy.is_shutdown()` return `True` if this node gets the signal to shutdown.
- The other way to simply shut this node down is by pressing `Ctrl-c` on the console that this node is executing.

talker.py

```
hello_str = "hello world %s" % rospy.get_time()
```

- Create a string named `hello_str`.
- `rospy.get_time()` returns the current time.

```
rospy.loginfo(hello_str)
```

- Print the message on the console screen that this node is executing.
- Store the message into node's log file
- Can also be viewed from any console using `rqt_console` command.

talker.py

```
        pub.publish(hello_str)
```

- Publish the message

```
        rate.sleep()
```

- Put this node to sleep for 0.1 second (10hz).

```
if __name__ == '__main__':
    try:
        talker()
    except rospy.ROSInterruptException:
        pass
```

- Interrupt may be generated by `rospy.Rate.sleep()`

Subscriber

- A **subscriber** subscribes to receive data from topics
- A subscriber has to have a name (for debugging purpose)
- A topic is a **named** bus (need a name as well)
- A topic must have a type associated with it (strongly typed)
- Do not need to care who are publishers (anonymous)

listener.py

```
#!/usr/bin/env python

import rospy
from std_msgs.msg import String

def callback(data):
    rospy.loginfo(rospy.get_caller_id() + "I heard %s", data.data)

def listener():
    rospy.init_node('listener', anonymous=True)
    rospy.Subscriber("chatter", String, callback)
    rospy.spin()

if __name__ == '__main__':
    listener()
```

Do not forget to make this file executable (`chmod +x talker.py`)

listener.py

```
rospy.init_node('listener', anonymous=True)
```

- Initialize this node with the name `listener`
- Should be anonymous

```
rospy.Subscriber("chatter", String, callback)
```

- Subscribes to the topic named `chatter`.
- The type of message is `String` (must be the same as the publisher)
- Every time a message is received, call the function `callback()`

```
rospy.spin()
```

- Keeps the node from exiting until the node has been shutdown.

The callback() Function

- This callback() function will be called every time it receives a message on the topic chatter:

```
def callback(data):
    rospy.loginfo(rospy.get_caller_id() + "I heard %s", data.data)
```

  - The argument data has type std_msgs/String
    - A data of this type consists of one component of type string named data
  - To access the actual string, we use data.data like C structure

`messenger.py`

- Sometimes your ROS node may behave like a middle man
  - Subscribes to a topic to receive data
  - Processes data
  - Publishes new data to a new topic
- Example: Blob Detection
  - Subscribes to a topic to receive a series of image
  - For each image received, find blobs of a specific color, and calculate the middle point of each blob
  - Publishes an array of coordinates of blobs
- This type of node subscribes as well as publishes data
- Example:
  - `messenger` node subscribe to the topic `chatter1` and publish the modified message to the topic `chatter2`

messenger.py

```
#!/usr/bin/env python

import rospy
from std_msgs.msg import String

pub = rospy.Publisher('chatter2', String, queue_size=10)

def chatterCallback(data):
    global pub
    messenger_str = "Messenger heard %s" % data.data
    pub.publish(messenger_str);

def messenger():
    rospy.init_node('messenger', anonymous=True)
    rospy.Subscriber("chatter1", String, chatterCallback)
    rospy.spin()

if __name__ == '__main__':
    messenger()
```

Notes about Distributed Computing

- ROS environment is similar to distributed environment.
  - Requires a number of processes to work together to finish a job.
  - If a process dies, the whole thing will not work properly.
  - We need to make sure that each process is alive until the job is done.
- Generally, if a program finish, its process dies (exit).
- In ROS, there are two main methods to keep a process alive.
  - Loop until a job is done.

```
while not rospy.is_shutdown():
    :
```

  - Use `rospy.spin()` which is almost the same as

```
while not rospy.is_shutdown():
    pass
```

Loop vs `rospy.spin()`

- You should use either loop or `rospy.spin()` **but not both** in the same program.
- Use loop when your program has to continuously checking and doing something.
  - Continuously publishes messages.
  - Continuously monitors variable values.
- Use `rospy.spin()` when you program can go to sleep until something happen.
  - Process an image or data whenever it is available (subscriber), publish just one message, and go back to sleep.
- Often time, the choice of loop vs `rospy.spin()` is not clear.

Cleanup

- A process (node) can be killed by pressing `Ctrl-c` or receiving the kill signal from ROS master
- Sometime, a process needs to clean up before exit.
  - If robot is currently moving, stop it first and exit
  - Otherwise, robot may keep moving non-stop
- Cleanup can be done by registering shutdown hooks:

```
def cleanup():
    # stop the robot

rospy.on_shutdown(cleanup)
```

  When the program receive the kill signal, it will call the function `cleanup()` first and then exit

- A node can manually shut itself down.

```
rospy.signal_shutdown("I am done.")
```

About Callback Functions

- Recall the `listener` node.

```
#!/usr/bin/env python

import rospy
from std_msgs.msg import String

def callback(data):
    rospy.loginfo(rospy.get_caller_id() + "I heard %s", data.data)

def listener():
    rospy.init_node('listener', anonymous=True)
    rospy.Subscriber("chatter", String, callback)
    rospy.spin()

if __name__ == '__main__':
    listener()
```

- The function `callback()` will be called every time a message from the topic `chatter` is received.

About Callback Functions

- **Rule of Thumb**:
  - A callback function should be as short as possible
  - If you need to process a large data (e.g., image), the processing part should not be in the callback function
    - The callback should simply update global variables and let the program process it when it is ready to process.

```
:
from copy import deepcopy

image = None

def aCallback(data):
    image = data

def mainProgram():
    :
    while not rospy.is_shutdown():
        :
        copyImage = deepcopy(image)
        # process copyImage
        :
```

rostopic

- `rostopic` is a tool related to topics in ROS environment
- It allows us to perform the following:
  - List the current active topics
  - Show messages publish to a specific topic on screen
  - Publish a message to a specific topic
  - Show information about a specific topic
  - Show type of a specific topic
- Suppose we run the `talker` and `listener` node using the following commands in three separate console screen:

```
roscore
```

```
rosrun mypackage talker.py
```

```
rosrun mypackage listener.py
```

rostopic

- Use

```
rostopic list
```

to list the active topics on the console screen.

```
/chatter
/rosout
/rosout_agg
```

- Use

```
rostopic echo chatter
```

to show messages published to the topic `chatter`

```
data: hello from talker 1463668746.93
---
data: hello from talker 1463668747.03
---
data: hello from talker 1463668747.23
---
data: hello from talker 1463668747.33
---
:
```

rostopic

- Use

```
rostopic info chatter
```

to show information about the topic chatter

```
Type: std_msgs/String

Publishers:
 * /talker_4544_1463668570440 (http://r2-d2:56659/)

Subscribers:
 * /listener_4723_1463668575785 (http://r2-d2:35408)
```

- Use

```
rostopic type chatter
```

to show the type of the topic chatter

```
std_msgs/String
```

- To publish the string "Blah Blah Blah" to the topic chatter

```
rostopic pub /chatter std_msgs/String "Blah Blah Blah"
```

Workspace Directory

- Your workspace directory is

```
/home/[username]/cs1567/src/mypackage
```

- All Python program should be in

```
/home/[username]/cs1567/src/mypackage/scripts
```

  or any subdirectory under the above directory.

- To quickly change to `../mypackage` directory, you can simply type

```
roscd my[Tab]
```

  where `[Tab]` is the `Tab` button on your keyboard.